

# Databases Fundamentals

Die Grundlage für persistente Daten und stabile Geschäftsdomänen

# SQL vs. NoSQL: Die zwei Welten

Datenbanken werden grundlegend in **relationale (SQL)** und **nicht-tabellarische (NoSQL)** Systeme unterteilt. Im echten Entwickleralltag überwiegt meist der relationale Ansatz, da sich Business-Logiken hervorragend über feste Beziehungen abbilden lassen.

## SQL (Relational)

- Basiert auf strikten **Beziehungen**
- Festes, vordefiniertes **Schema**
- Starke Datenintegrität
- Schwieriger horizontal zu skalieren
- **Tech-Stack:** PostgreSQL, MySQL

## NoSQL (Non-Tabular)

- Keine starre Form (**Schemalos**)
- Eingeschränkte Beziehungslogik
- Sehr hohe Performance
- Hervorragend horizontal skalierbar
- **Tech-Stack:** MongoDB, Neo4j

# Wann nutzt man was?

Die Wahl des Datenbanksystems hängt von den Anforderungen der Anwendung ab:

| Kriterium | SQL-Datenbanken                   | NoSQL-Datenbanken                  |
|-----------|-----------------------------------|------------------------------------|
| Fokus     | Komplexe Beziehungen wichtig      | Maximale Performance & Durchsatz   |
| Schema    | Benötigt ein verlässliches Schema | Flexibel für unstrukturierte Daten |
| Beispiele | E-Commerce-Systeme, Banking       | CMS, Echtzeitdaten, Log-Storage    |

# SQL – Was ist das?

Im Kern ist SQL eine strukturierte Tabelle mit festen Spalten.

Unsere Beispieltabelle: `users`. Alle folgenden Beispiele arbeiten mit derselben Tabelle.

| id | first_name | age | status   |
|----|------------|-----|----------|
| 1  | Anna       | 22  | active   |
| 2  | Ben        | 16  | active   |
| 3  | Clara      | 30  | inactive |

Jede Zeile repräsentiert einen Datensatz.

Jede Spalte beschreibt ein bestimmtes Attribut.

# CRUD – Die vier Grundoperationen

Jede Anwendung arbeitet permanent mit diesen vier Aktionen:

| Operation | Bedeutung           |
|-----------|---------------------|
| CREATE    | Daten erstellen     |
| READ      | Daten lesen         |
| UPDATE    | Daten aktualisieren |
| DELETE    | Daten löschen       |

# ● CREATE – Daten erstellen

## JavaScript

```
const users = [  
  { id: 1, first_name: "Anna", age: 22, status: "active" },  
  { id: 2, first_name: "Ben", age: 16, status: "active" },  
  { id: 3, first_name: "Clara", age: 30, status: "inactive" },  
];  
  
users.push({  
  id: 4,  
  first_name: "Tom",  
  age: 20,  
  status: "active",  
});
```

# SQL

```
INSERT INTO users (id, first_name, age, status)
VALUES (4, 'Tom', 20, 'active');
```

➔ SQL macht exakt dieselbe Operation persistent in der Datenbank.

# ● READ – Daten lesen

## JavaScript

```
const users = [  
  { id: 1, first_name: "Anna", age: 22, status: "active" },  
  { id: 2, first_name: "Ben", age: 16, status: "active" },  
  { id: 3, first_name: "Clara", age: 30, status: "inactive" },  
];  
  
// Hole Namen aller User  
const result = users.map((u) => ({ first_name: u.first_name }));
```

➡ `map()` = bestimmte Felder zurückgeben



# SQL

Die Anatomie einer Standard-SQL-Abfrage besteht immer aus diesen Klauseln:

- **SELECT**: Bestimmt, **welche Spalten** zurückgegeben werden.
- **FROM**: Bestimmt, aus **welcher Tabelle** gelesen wird.

```
-- Hole Namen aller User  
SELECT first_name FROM users
```

oder alle Spalten

```
SELECT * FROM users
```

# SQL Query Basics

Die Anatomie einer SQL-Abfrage:

| Klausel  | Bedeutung                                   |
|----------|---------------------------------------------|
| SELECT   | Welche Spalten sollen zurückgegeben werden? |
| FROM     | Aus welcher Tabelle wird gelesen?           |
| WHERE    | Welche Zeilen sollen gefiltert werden?      |
| ORDER BY | Wie sollen Ergebnisse sortiert werden?      |
| LIMIT    | Wie viele Ergebnisse sollen zurückkommen?   |

# Beispiel

```
SELECT first_name  
FROM users  
WHERE age > 18  
ORDER BY age DESC  
LIMIT 5;
```

# Select mit Constraints

```
const result = users
  .filter((u) => u.age > 18 && u.status === "active")
  .map((u) => ({ first_name: u.first_name }));
```

```
SELECT first_name
FROM users
WHERE age > 18 AND status = 'active';
```

➔ `filter()` = Datensätze auswählen

# ● UPDATE – Daten aktualisieren

## JavaScript

```
const users = [  
  { id: 1, first_name: "Anna", age: 22, status: "active" },  
  { id: 2, first_name: "Ben", age: 16, status: "active" },  
  { id: 3, first_name: "Clara", age: 30, status: "inactive" },  
];  
  
const updatedUsers = users.map((u) => (u.id === 2 ? { ...u, age: 17 } : u));
```

➡ Datensatz gezielt verändern

# SQL

```
UPDATE users  
SET age = 17  
WHERE id = 2;
```

# ● DELETE – Daten löschen

## JavaScript

```
const users = [  
  { id: 1, first_name: "Anna", age: 22, status: "active" },  
  { id: 2, first_name: "Ben", age: 16, status: "active" },  
  { id: 3, first_name: "Clara", age: 30, status: "inactive" },  
];  
  
const filteredUsers = users.filter((u) => u.id !== 2);
```

➡ Alles behalten außer dem Treffer

# SQL

```
DELETE FROM users  
WHERE id = 2;
```



# Zusammenfassung

SQL ist keine neue Denkweise

Das kennt ihr bereits aus JavaScript

| SQL    | JavaScript                     |
|--------|--------------------------------|
| INSERT | <code>push()</code>            |
| SELECT | <code>map()</code>             |
| WHERE  | <code>filter()</code>          |
| UPDATE | <code>map() + condition</code> |
| DELETE | <code>filter()</code>          |

# Beispiel

```
SELECT first_name  
FROM users  
WHERE age > 18;
```

## JavaScript Denkweise dazu

```
users  
  .filter((u) => u.age > 18)  
  .map((u) => ({  
    first_name: u.first_name,  
  }));
```

# Warum Datenbanken trotzdem wichtig sind

JavaScript Arrays:

- sind nur im RAM
- verschwinden beim Neustart
- sind nicht für Millionen Datensätze optimiert

Datenbanken bieten:

- Persistenz
- Performance
- Sicherheit
- Skalierung
- gemeinsame Datenquelle

# Fazit

- SQL = strukturierte relationale Daten
- CRUD = Grundlage jeder Anwendung
- JavaScript Arrays = perfekte Vorbereitung auf SQL
- Datenbanken speichern Business-Logik dauerhaft
- Datenabfragen sind reine Logik

# Beziehungen zwischen Tabellen

Relationale Datenbanken werden mächtig, sobald Tabellen miteinander verbunden werden.

# Beispiel: Users & Orders

users

| id | name |
|----|------|
| 1  | Anna |
| 2  | Ben  |

orders

| id | user_id | product |
|----|---------|---------|
| 1  | 1       | Laptop  |
| 2  | 1       | Mouse   |

# ER-Diagramm (Entity Relationship)

users

-----

id (PK)

name

1 ————— ∞

orders

-----

id (PK)

user\_id (FK)

product

## Bedeutung

- Ein User kann viele Orders besitzen
- Jede Order gehört genau einem User

# JOIN – Tabellen verbinden

Mit `JOIN` kombiniert SQL Daten aus mehreren Tabellen.

```
SELECT users.name, orders.product
FROM users
JOIN orders
ON users.id = orders.user_id;
```

## Ergebnis

| name | product  |
|------|----------|
| Anna | Laptop   |
| Anna | Mouse    |
| Ben  | Keyboard |



SQL ist keine Magie

| Daten abfragen ist **Logik**.

Das Tool entscheidet nur über die Syntax.

# Wichtig für echte Anwendungen

Datenbanken sind nicht nur Speicher.

Sie modellieren:

- Geschäftslogik
- Beziehungen
- Zustände
- Historie
- Sicherheit
- Konsistenz

# Moderne Architektur

Typischer Stack:

Frontend



Backend/API



Database

Die Datenbank ist die zentrale Wahrheit des Systems.

# Zusammenfassung

- SQL = strukturierte relationale Daten
- NoSQL = flexible hochskalierbare Daten
- CRUD = Basis jeder Anwendung
- JOINS verbinden Geschäftslogik
- Datenabfragen sind reine Logik
- Gute Datenmodelle sind wichtiger als Frameworks

Daten abfragen ist **Logik**.

| Das Tool entscheidet nur über die Syntax.



# Aufgabe:

- **SQL Practice:**
  - Sie müssen SQL-Queries nicht zwingend auswendig im Schlaf schreiben können, aber das Verständnis für relationale Operationen ist Pflicht.
  - **Fokus im Interview:** Beherrschen Sie `INNER JOIN` und `LEFT JOIN` sicher auf einfachem bis mittlerem Schwierigkeitsgrad.
- **NoSQL (MongoDB) Practice:**
  - Machen Sie sich mit dokumentenorientierten Abfragen vertraut.
  - Nutzen Sie Tools wie `humongous.io`, um grundlegende CRUD-Queries gegen eine Live-NoSQL-Datenbank auszuführen und sich auf typische Screening-Fragen vorzubereiten.

# Das SQL-Datenbankschema

In der relationalen Welt ist das **Schema** das Fundament der Datenintegrität. Es beschreibt exakt die Struktur und die Datentypen aller Entitäten.

*"Ein Schema macht Datenbankabfragen verlässlich, typsicher und vorhersagbar."*

- **Sicherer Vertrag:** Änderungen am Datenmodell erfordern explizite Migrationen (Hinzufügen von Tabellen, Spalten oder neuen Fremdschlüssel-Beziehungen).
- **Visuelle Modellierung:** Beziehungen zwischen Entitäten (z. B. wie ein `actor` mit `movies` verknüpft ist oder wo `movie_reviews` andocken) werden in Entity-Relationship-Diagrammen (ERD) strukturiert.

# Daten halten. Struktur bewahren.

Die Wahl der Datenbank bestimmt das Schicksal der Skalierung.

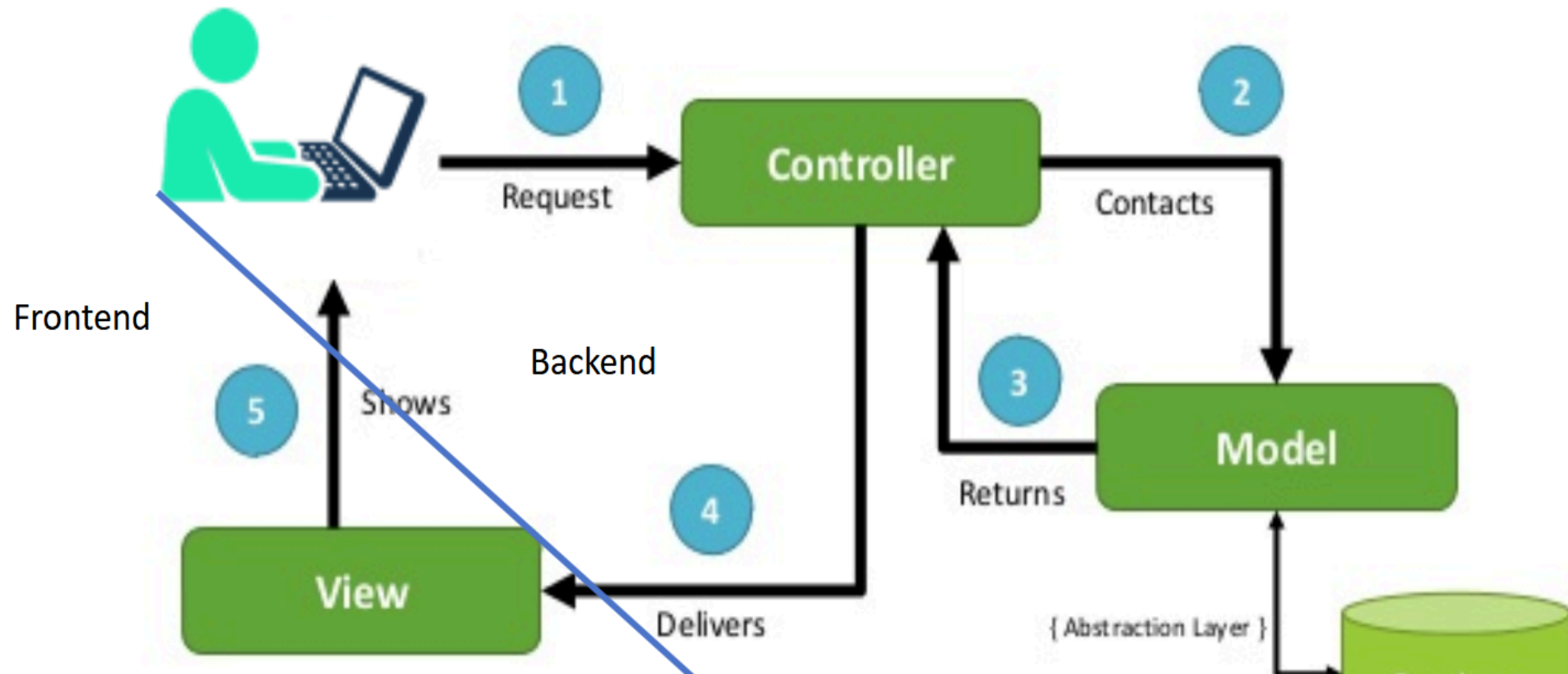
PostgreSQL | MongoDB | ERD | Schema Design



| ORM

# Wrap it up

## MVC



- Model: Repräsentation der Daten
- View: Darstellung der Daten für den Benutzer
- Controller: Vermittler zwischen Model und View, verarbeitet Benutzereingaben; Geschäftslogik
- Früher war View ein HTML-Template, heute ist es oft die REST API. Zusätzlich gibt es dann eine Single Page Application (SPA) als Frontend für Nutzer.
  - zum Beispiel ASP.NET MVC, Spring MVC, Ruby on Rails